

Exercice 1 : Allocation dynamique de la mémoire Évaluer la compréhension des concepts fondamentaux

1. Tableaux et Boucles
 - a. Écrivez un programme en C qui prend en entrée un tableau d'entiers de taille N et affiche la somme de tous les éléments du tableau.
2. Pointeurs et Fonctions
 - a. Déclarez une fonction en C qui prend deux pointeurs vers des entiers en paramètres et les échange.
 - b. Utilisez cette fonction pour échanger les valeurs de deux variables entières.
3. Chaînes de caractères
 - a. Écrivez un programme en C++ qui prend une chaîne de caractères en entrée et affiche la fréquence de chaque lettre dans la chaîne.
4. Structures et Tableaux
 - a. Définissez une structure Personne avec des champs pour le nom, l'âge et la ville de résidence.
 - b. Écrivez un programme en C++ qui utilise un tableau de structures Personne pour stocker les informations de plusieurs personnes et affiche le nom de la personne la plus âgée.
5. Entrées/Sorties
 - a. Écrivez un programme en C++ qui lit un fichier texte contenant des nombres entiers (un nombre par ligne), calcule leur somme et écrit le résultat dans un autre fichier.

Exercice 2 : Allocation dynamique de la mémoire

1. Écrivez une fonction qui crée et renvoie un tableau aléatoire d'entiers.

`int* creerTableauAleatoire(int taille)`
2. Développez une procédure pour afficher un tableau d'entiers passé en paramètre.

`void afficherTableau(int* tableau, int taille)`
3. Rédigez le code d'une fonction capable de retourner la différence entre deux tableaux d'entiers passés en paramètre.

`int* differenceTableaux(int* tableau1, int* tableau2, int taille1, int taille2, int& nouvelleTaille)`
4. Enfin, concevez un programme principal qui invite l'utilisateur à saisir deux entiers, m et n, puis à générer deux tableaux d'entiers de tailles respectives m et n. Affichez les deux tableaux à l'écran, calculez et affichez la différence entre les deux tableaux.

Exercice 3 : Structure de données Gestion des pixels et des images

1. Créer une structure représentant un pixel avec trois valeurs RGB, et une structure pour une image composée d'une matrice de pixels.
 - a. Créez une structure Pixel qui contient les informations suivantes :
 - i. Composantes rouge, verte et bleue (RGB)

```
struct Pixel {  
    int rouge;
```

```
int vert;  
int bleu;  
};
```

- b. Créez une structure Image qui contient les informations suivantes :
- Une matrice de pixels représentant une image.
 - Dimensions de l'image (largeur et hauteur).

```
struct Image {  
    Pixel pixels[MAX_LARGEUR][MAX_HAUTEUR];  
    int largeur;  
    int hauteur;  
};
```

2. Écrivez une fonction qui retourne une structure Pixel avec des valeurs aléatoires.

```
Pixel creerPixelAleatoire()
```

3. Écrivez une fonction qui prend en paramètres la largeur et la hauteur et retourne une structure Image avec une matrice de pixels de la taille spécifiée.

```
Image creerImage(int largeur, int hauteur)
```

4. Écrivez une fonction qui affiche les composantes RGB de chaque pixel de l'image.

```
void afficherImage(const Image& image)
```

5. Proposer un programme principal pour tester les fonctions.

Correction 1 :

```
#include <stdio.h>
int main() {
    int N;
    printf("Entrez la taille du tableau : ");
    scanf("%d", &N);
    int tableau[N];
    int somme = 0;
    printf("Entrez les éléments du tableau :\n");
    for (int i = 0; i < N; ++i) {
        scanf("%d", &tableau[i]);
        somme += tableau[i];
    }
    printf("La somme des éléments du tableau est : %d\n", somme);
    return 0;
}
```

```
#include <stdio.h>
void echanger(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    printf("Avant l'échange : x = %d, y = %d\n", x, y);
    echanger(&x, &y);
    printf("Après l'échange : x = %d, y = %d\n", x, y);
    return 0;
}
```

```
#include <iostream>

// Fonction pour déterminer si un caractère est une lettre
bool estLettre(char caractere) {
    return (caractere >= 'a' && caractere <= 'z') || (caractere >= 'A' && caractere <= 'Z');
}

// Fonction pour convertir un caractère en minuscule
char toLowerCase(char caractere) {
    if (caractere >= 'A' && caractere <= 'Z') {
        return caractere + ('a' - 'A');
    }
    return caractere;
}

// Fonction pour calculer la fréquence des lettres dans une chaîne
void calculerFrequenceLettres(char chaine[], int frequence[]) {
    for (int i = 0; chaine[i] != '\0'; ++i) {
        if (estLettre(chaine[i])) {
            char lettreMinuscule = toLowerCase(chaine[i]);
            frequence[lettreMinuscule - 'a']++;
        }
    }
}
```

```

}

int main() {
    const int TAILLE_MAX = 1000;
    char chaine[TAILLE_MAX];

    std::cout << "Entrez une chaîne de caractères : ";
    std::cin.getline(chaine, TAILLE_MAX);

    const int ALPHABET_SIZE = 26;
    int frequence[ALPHABET_SIZE] = {0};

    calculerFrequenceLettres(chaine, frequence);

    std::cout << "Fréquence des lettres dans la chaîne :\n";
    for (int i = 0; i < ALPHABET_SIZE; ++i) {
        if (frequence[i] > 0) {
            char lettre = 'a' + i;
            std::cout << lettre << " : " << frequence[i] << " fois\n";
        }
    }

    return 0;
}

```

```

#include <iostream>

struct Personne {
    std::string nom;
    int age;
    std::string ville;
};

int main() {
    const int taille = 3;
    Personne personnes[taille];

    for (int i = 0; i < taille; ++i) {
        std::cout << "Entrez le nom de la personne " << i + 1 << " : ";
        std::cin >> personnes[i].nom;
        std::cout << "Entrez l'âge de la personne " << i + 1 << " : ";
        std::cin >> personnes[i].age;
        std::cout << "Entrez la ville de résidence de la personne " << i + 1 << " : ";
        std::cin >> personnes[i].ville;
    }

    int indexPersonneAgee = 0;
    for (int i = 1; i < taille; ++i) {
        if (personnes[i].age > personnes[indexPersonneAgee].age) {
            indexPersonneAgee = i;
        }
    }
}

```

```

    std::cout << "La personne la plus âgée est : " << personnes[indexPersonneAgee].nom <<
    std::endl;

    return 0;
}

#include <iostream>
#include <fstream>

int main() {
    std::ifstream fichierEntree("entrees.txt");
    int nombre;
    int somme = 0;

    while (fichierEntree >> nombre) {
        somme += nombre;
    }

    std::ofstream fichierSortie("sortie.txt");
    fichierSortie << "La somme des nombres du fichier d'entrée est : " << somme << std::endl;

    fichierEntree.close();
    fichierSortie.close();

    return 0;
}

```

Correction 2 : Allocation dynamique de la mémoire

```

#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;
// Fonction pour créer et renvoyer un tableau aléatoire d'entiers
int* creerTableauAleatoire(int taille) {
    int* tableau = new int[taille];
    srand(time(0)); // Initialisation du générateur de nombres aléatoires avec la seed actuelle
    for (int i = 0; i < taille; ++i) {
        tableau[i] = rand() % 100; // Génère des entiers aléatoires entre 0 et 99
    }
    return tableau;
}
// Procédure pour afficher un tableau d'entiers passé en paramètre
void afficherTableau(int* tableau, int taille) {
    for (int i = 0; i < taille; ++i) {
        cout << tableau[i] << " ";
    }
    cout << endl;
}
// Fonction pour retourner la différence entre deux tableaux d'entiers passés en paramètre
int* differenceTableaux(int* tableau1, int* tableau2, int taille1, int taille2, int& nouvelleTaille) {
    int* difference = new int[taille1];
    nouvelleTaille = 0;
}

```

```

    for (int i = 0; i < taille1; ++i) {
        bool presentDansTableau2 = false;
        for (int j = 0; j < taille2; ++j) {
            if (tableau1[i] == tableau2[j]) {
                presentDansTableau2 = true;
                break;
            }
        }
        if (!presentDansTableau2) {
            difference[nouvelleTaille] = tableau1[i];
            nouvelleTaille++;
        }
    }
    return difference;
}

int main() {
    // Demande à l'utilisateur de saisir deux entiers, m et n
    int m, n;
    cout << "Entrez la taille du premier tableau (m): ";
    cin >> m;
    cout << "Entrez la taille du deuxième tableau (n): ";
    cin >> n;
    // Génère deux tableaux d'entiers de tailles respectives m et n
    int* tableau1 = creerTableauAleatoire(m);
    int* tableau2 = creerTableauAleatoire(n);
    // Affiche les deux tableaux à l'écran
    cout << "Premier tableau: ";
    afficherTableau(tableau1, m);
    cout << "Deuxième tableau: ";
    afficherTableau(tableau2, n);
    // Calcule et affiche la différence entre les deux tableaux
    int nouvelleTaille;
    int* difference = differenceTableaux(tableau1, tableau2, m, n, nouvelleTaille);

    cout << "Différence entre les deux tableaux: ";
    afficherTableau(difference, nouvelleTaille);
    // Libère la mémoire allouée dynamiquement
    delete[] tableau1;
    delete[] tableau2;
    delete[] difference;

    return 0;
}

```

Correction 3 : Structure de données Gestion des pixels et des images

```

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int MAX_LARGEUR = 10;

```

```

const int MAX_HAUTEUR = 10;

struct Pixel {
    int rouge;
    int vert;
    int bleu;
};

struct Image {
    Pixel pixels[MAX_LARGEUR][MAX_HAUTEUR];
    int largeur;
    int hauteur;
};

Pixel creerPixelAleatoire() {
    Pixel nouveauPixel;
    nouveauPixel.rouge = rand() % 256; // Valeurs entre 0 et 255 inclus
    nouveauPixel.vert = rand() % 256;
    nouveauPixel.bleu = rand() % 256;
    return nouveauPixel;
}

Image creerImage(int largeur, int hauteur) {
    Image nouvelleImage;
    nouvelleImage.largeur = largeur;
    nouvelleImage.hauteur = hauteur;
    for (int i = 0; i < largeur; ++i) {
        for (int j = 0; j < hauteur; ++j) {
            nouvelleImage.pixels[i][j] = creerPixelAleatoire();
        }
    }
    return nouvelleImage;
}

void afficherImage(const Image& image) {
    for (int i = 0; i < image.largeur; ++i) {
        for (int j = 0; j < image.hauteur; ++j) {
            cout << "(" << image.pixels[i][j].rouge << ", "
                << image.pixels[i][j].vert << ", "
                << image.pixels[i][j].bleu << ") ";
        }
        cout << endl;
    }
}

int main() {
    Image monImage = creerImage(5, 3);
    afficherImage(monImage);
    return 0;
}

```